Example — Early Error Detection

```
// sample.ts
let count: number = 0;
count = "oops";   // Error: Type 'string' is not assignable to type 'number'

$ node sample.ts
// Expected output: 0
```

TypeScript flags the type mismatch at compile time itself — before the program is ever run — preventing a bug from reaching production.

Example — TypeScript in an Angular Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  title: string = 'hello-world-app';
}
```

- @Component({...}) is a decorator — a TypeScript feature — that marks the class as an Angular component and links it to its template.
- title: string = ... declares a typed property; assigning a non-string value later would be flagged immediately by TypeScript.

Plugging the New Component into the App

```
// app.component.ts (root component)


import { Component } from '@angular/core';
import { HelloWorldComponent } from './hello-world/hello-world.component';

@Component({
  selector: 'app-root',
  imports: [HelloWorldComponent],
  template: `
    <h2>Simple App Demo</h2>
    <app-hello-world></app-hello-world>
  `
})
export class AppComponent { }
```

- AppComponent is the root component every Angular app starts from — it is bootstrapped automatically when the app runs.
- <app-hello-world></app-hello-world> is the selector of HelloWorldComponent. Angular replaces this tag with the component's rendered output, proving components are reusable, drop-in building blocks.

```
// hello-world.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-world.component.html',
  styleUrls: ['./hello-world.component.css']
})
export class HelloWorldComponent { }

<!-- hello-world.component.html -->
<h3>Hello World</h3>
<h4>This is Hello World Component</h4>
```

- HelloWorldComponent's selector: 'app-hello-world' is exactly what makes <app-hello-world></app-hello-world> work inside AppComponent's template — Angular replaces that tag with this component's rendered HTML.
- AppComponent imports HelloWorldComponent from './hello-world/hello-world.component' and lists it in its own imports: [HelloWorldComponent] array (standalone component syntax) — without this import, Angular would not recognize <app-hello-world> as a valid tag in AppComponent's template.

Hands-On: Creating a Component

```
ng generate component hello-world
# or the short form:
ng g c hello-world

// hello-world.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-world.component.html',
  styleUrls: ['./hello-world.component.css']
})
export class HelloWorldComponent { }

<!-- hello-world.component.html -->
<h3>Hello World</h3>
<h4>This is Hello World Component</h4>
```

The component's selector (app-hello-world) can then be used anywhere in the app, like any other HTML tag, and Angular will render this component wherever the tag appears — this is what makes components reusable, drop-in building blocks.

2. Attribute Directive

Definition: An attribute directive is an Angular directive that is applied as an attribute to an **existing** HTML element to change its appearance or add behavior.

Built-in Attribute Directive — ngClass

```
// typescript
import { Component } from '@angular/core';
import { NgClass } from '@angular/common';
@Component({
 selector: 'app-root',
 imports: [NgClass],
 template: `
  <h2>ngClass Example</h2>
  <p [ngClass]="{'highlight': isHighlighted}">
   Hello Students!
  </p>
  <button (click)="isHighlighted = !isHighlighted">
   Change Color
  </button>
 `,
 styles: `
  .highlight {
   color: red;
   font-size: 25px;
  }
 `
})
export class App {
 isHighlighted = false; }
```

- [ngClass]="{'highlight': isHighlighted}" conditionally applies the CSS class highlight to the <p> element whenever isHighlighted is true.
- Clicking **Change Color** toggles isHighlighted, which adds/removes the highlight class — turning the text red and larger, since Angular re-evaluates the binding automatically.
- **Output:** Initially "Hello Students!" is plain; after clicking "Change Color," the text turns red and bigger.

Built-in Attribute Directive — ngStyle

```
//typescript
import { Component } from '@angular/core';
import { NgStyle } from '@angular/common';
@Component({
 selector: 'app-root',
 imports: [NgStyle],
 template: `
  <h2>ngStyle Example</h2>
  <p [ngStyle]="{
   'color': isHighlighted ? 'red' : 'black',
   'font-size': isHighlighted ? '30px' : '20px'
  }">
   Hello Students!
  </p>

  <button (click)="isHighlighted = !isHighlighted">
   Change Style
  </button>
 `
})
export class App {
 isHighlighted = false; }
```

- [ngStyle]="{...}" binds an object of CSS style properties directly to the element — here color and font-size change based on the isHighlighted condition, using inline ternary expressions.
- Unlike ngClass (which toggles a predefined CSS class), ngStyle lets you set **individual style properties directly**, computed dynamically in the template.

a) *ngIf

```
//typescript
@Component({
 selector: 'app-root',
 imports: [NgIf],
 template: `
  <h2>Hello World</h2>
  <button (click)="isVisible = !isVisible">
   Show / Hide
  </button>

  <p *ngIf="isVisible">
   Hello Students!
  </p>
 `
})
export class App {
 isVisible = true;
}
```

- *ngIf="isVisible" creates the element in the DOM only when isVisible is true; when false, the element is **completely removed** from the DOM (not just hidden).
- Clicking **Show/Hide** toggles isVisible, which adds or removes the Hello Students! element from the page in real time.

b) *ngFor

```
//typescript
import { Component } from '@angular/core';
import { NgFor } from '@angular/common';


@Component({
 selector: 'app-root',
 imports: [NgFor],
 template: `
  <h2>Student List</h2>
  <ul>
   <li *ngFor="let student of students">
    {{ student }}
   </li>
  </ul>
 `
})
export class App {
 students = ['Anu', 'Ravi', 'Priya', 'Kiran'];  }
```

c) *ngSwitch

```
//typescript
@Component({
 selector: 'app-root',
 imports: [NgSwitch, NgSwitchCase, NgSwitchDefault],
 template: `
  <h2>Student Grade</h2>

  <div [ngSwitch]="grade">
   <p *ngSwitchCase="'A'">Excellent</p>
   <p *ngSwitchCase="'B'">Very Good</p>
   <p *ngSwitchCase="'C'">Good</p>
   <p *ngSwitchDefault>Invalid Grade</p>
  </div>
 `
})
export class App {
 grade = 'B';
}
```

- [ngSwitch]="grade" evaluates the grade property once, and Angular checks it against each *ngSwitchCase value.
- Only the matching is rendered in the DOM — here, since grade = 'B', only Very Good appears.
- *ngSwitchDefault renders as a fallback only if none of the *ngSwitchCase values match.
- **Output:** "Very Good" is displayed on the page.

Date Pipe — Example

```
import { Component } from '@angular/core';
import { DatePipe } from '@angular/common';
```

```
@Component({
 selector: 'app-root',
 imports: [DatePipe],
 template: `
  <p>Default: {{ today | date }}</p>
  <p>Short Date: {{ today | date:'shortDate' }}</p>
  <p>Custom: {{ today | date:'dd/MM/yyyy' }}</p>
 `
})
export class App {
 today = new Date();
}
```

- today | date applies the DatePipe with its default format.
- today | date:'shortDate' passes 'shortDate' as an argument, producing a compact format like 8/20/26.
- The today variable in the class itself is never modified — only its displayed representation changes.

Example:

```
// typescript
import { Component } from '@angular/core';
import { DatePipe } from '@angular/common';
@Component({
 selector: 'app-root',
 imports: [DatePipe],
 template: `
  <h2>Date Pipe Example</h2>
  <p>Default: {{ today | date }}</p>
  <p>Short Date: {{ today | date:'shortDate' }}</p>
  <p>Medium Date: {{ today | date:'mediumDate' }}</p>
  <p>Long Date: {{ today | date:'longDate' }}</p>
```

```
  <p>Full Date: {{ today | date:'fullDate' }}</p>
  <p>Custom: {{ today | date:'dd/MM/yyyy' }}</p>
  <p>Custom: {{ today | date:'dd-MM-yyyy' }}</p>
  <p>Custom: {{ today | date:'dd MMMM yyyy' }}</p>
 `
})
export class App {
 today = new Date();
}
```

Output:

Default: Aug 20, 2026

Short Date: 8/20/26

Medium Date: Aug 20, 2026

Long Date: August 20, 2026

Full Date: Thursday, August 20, 2026

Custom: 20/08/2026

Custom: 20-08-2026

Custom: 20 August 2026

2. JSON Pipe

Example:

```
//typescript
import { Component } from '@angular/core';
import { JsonPipe } from '@angular/common';
```

```
@Component({
 selector: 'app-root',
 imports: [JsonPipe],
 template: `
  <h2>JSON Pipe Example</h2>
  <p>Without JSON pipe:</p>
  <p>{{ student }}</p>
  <p>With JSON pipe:</p>
  <p>{{ student | json }}</p>
  <p>Only Name:</p>
  <p>{{ student.name }}</p>
 `
})
export class App {
 student = {
  name: 'Anu',
  age: 20
 };
}
```

Output:

Without JSON pipe: [object Object]

With JSON pipe: {"name":"Anu","age":20}

Only Name: Anu

- Without the json pipe, Angular cannot display an object directly in the template — it prints [object Object].
- The json pipe converts the object into its **JSON string representation**, making it readable — useful for debugging.

- Individual properties (like student.name) can still be accessed and displayed directly without the pipe.

3. Decimal, Currency and Percent Pipe

Example:

```
//typescript
import { Component } from '@angular/core';
import { DecimalPipe, CurrencyPipe, PercentPipe } from '@angular/common';
@Component({
 selector: 'app-root',
 imports: [DecimalPipe, CurrencyPipe, PercentPipe],
 template: `
  <h2>Number, Currency and Percent Pipes</h2>

  <p>Number: {{ number | number }}</p>
  <p>Currency: {{ price | currency:'INR' }}</p>
  <p>Percentage: {{ percentage | percent }}</p>
 `
})
export class App {
 number = 1234567.89;
 price = 2500;
 percentage = 0.75;
}
```

Output:

Number: 1,234,567.89

Currency: ₹2,500.00

Percentage: 75%

- number pipe formats a plain number with grouping separators (commas) and decimal places.
- currency:'INR' formats the value as Indian Rupee currency, adding the ₹ symbol and two decimal places.
- percent converts a decimal fraction (e.g., 0.75) into a percentage display (75%).

4. UpperCase, LowerCase, TitleCase, and Slice Pipe

Example:

```
//typescript
import { Component } from '@angular/core';
import {
 UpperCasePipe,
 LowerCasePipe,
 TitleCasePipe,
 SlicePipe
} from '@angular/common';
@Component({
 selector: 'app-root',
 imports: [
  UpperCasePipe,
  LowerCasePipe,
  TitleCasePipe,
  SlicePipe
 ],
 template: `
  <h2>Angular Pipes Example</h2>
  <p>Original: {{ name }}</p>
  <p>Uppercase: {{ name | uppercase }}</p>
```

```
  <p>Lowercase: {{ name | lowercase }}</p>
  <p>Titlecase: {{ name | titlecase }}</p>
  <p>Slice: {{ name | slice:0:5 }}</p>
 `
})
export class App {
 name = 'hello students';
}
```

Output:

Original: hello students

Uppercase: HELLO STUDENTS

Lowercase: hello students

Titlecase: Hello Students

Slice: hello

- uppercase / lowercase convert the entire string to upper or lower case.
- titlecase capitalises the first letter of every word.
- slice:0:5 extracts a portion of the string — here, characters from index 0 up to (but not including) index 5, giving "hello".

Example:

```
typescript
// parent.component.ts
export class ParentComponent {
 count = 10;
 increase() {
  this.count++;
 }
}
```

```
html
<!-- parent.component.html -->
<p>Parent Count: {{ count }}</p>
<button (click)="increase()">Increase</button>
<app-child [myCount]="count"></app-child>
```

```
// typescript
// child.component.ts
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';
export class ChildComponent implements OnChanges {
 @Input() myCount!: number;
 ngOnChanges(changes: SimpleChanges) {
  console.log("myCount changed");
  console.log(changes);
 }
}
```

- Every time the parent's count changes, Angular re-evaluates [myCount]="count" and calls ngOnChanges() in the child, passing a SimpleChanges object describing exactly what changed.

Example:

```
// typescript
import { Component, OnInit } from '@angular/core';
@Component({
  selector: 'app-root',
  template: `<h2>Hello Angular</h2>`
})
export class App implements OnInit {
  ngOnInit() {
    console.log('App component initialized');
  }
}
```

Example (Content Projection Setup):

```
//html
<!-- Usage -->
<app-child>
  <p>Hello</p>
</app-child>
html
<!-- child.component.html -->
<div>
  <h2>Child Component</h2>
  <ng-content></ng-content>
</div>
```

- The Hello passed between <app-child> tags is **projected** into the <ng-content> slot — ngAfterContentInit() fires right after this projected content is first initialized.

Example:

```
// typescript
// child.component.ts
import { Component, OnDestroy } from '@angular/core';
@Component({
 selector: 'app-child',
 template: `
  <h2>Child Component</h2>
  <p>I am the child component.</p>
 `
})
export class ChildComponent implements OnDestroy {
 ngOnDestroy() {
  console.log('Child component is destroyed');
 }
}
```

```
// typescript
// app.component.ts (parent)
export class AppComponent {
 showChild = true;
}
html
<!-- app.component.html -->
<h2>Parent Component</h2>
<button (click)="showChild = !showChild">Show / Hide Child</button>
@if (showChild) {
 <app-child></app-child>
}
```

- A component is **destroyed** when Angular removes it from the application/view.
- Clicking "Show / Hide Child" toggles showChild; when it becomes false, Angular removes <app-child> from the DOM, triggering ngOnDestroy() in the child and logging 'Child component is destroyed'.

Example: Calculator Service via DI

Step 1 — Create the service:

```
// calculator.service.ts
import { Injectable } from '@angular/core';

@Injectable({
 providedIn: 'root'
})
export class CalculatorService {
 findSqrRoot(num: number) {
  return Math.sqrt(num);
 }
}
```

Step 2 — Inject the service into a component (constructor is used to inject the service or dependency):

```
// simple-ui.component.ts
import { Component } from '@angular/core';
import { CalculatorService } from './calculator.service';

@Component({
 selector: 'app-simple-ui',
 template: `
  <input #num type="number">
  <button (click)="calculate(num.value)">Find Square Root</button>
  <p>Result: {{ result }}</p>
 `
})
```

```
export class SimpleUiComponent {
 result: number = 0;

 constructor(private calculator: CalculatorService) {}

 calculate(value: string) {
  this.result = this.calculator.findSqrRoot(Number(value));
 }
}
```

Explanation

- @Injectable({ providedIn: 'root' }) registers CalculatorService with Angular's DI system at the root level, making it available application-wide.
- constructor(private calculator: CalculatorService) {} — instead of writing new CalculatorService(), the component simply declares its dependency here; Angular's DI system automatically creates and supplies the instance.
- this.calculator.findSqrRoot(...) — the component asks the injected service to perform the calculation; the component's own job is only to receive user input and display the result.

Here's the same SimpleUiComponent, rewritten to use inject() instead of constructor injection:

```
//                                                                simple-ui.component.ts
import { Component, inject } from '@angular/core';
import { CalculatorService } from './calculator.service';

@Component({
 selector: 'app-simple-ui',
 template: `
  <input #num type="number">
  <button (click)="calculate(num.value)">Find Square Root</button>
  <p>Result: {{ result }}</p>
 `
})
export class SimpleUiComponent {
 result: number = 0;
```

```
 // Inject service without constructor
 private calculator = inject(CalculatorService);

 calculate(value: string) {
  this.result = this.calculator.findSqrRoot(Number(value));
 }
}
```

What changed:

- import { Component } from '@angular/core'; → now also imports inject.
- constructor(private calculator: CalculatorService) {} is **removed entirely**.
- private calculator = inject(CalculatorService); directly requests the CalculatorService instance from Angular's DI system at the point of declaration.
- calculate(value: string) stays exactly the same — it still calls this.calculator.findSqrRoot(...), since calculator refers to the same injected service instance either way.

Both versions behave identically at runtime — same shared CalculatorService instance (since it's registered with providedIn: 'root'), same output. Only the syntax used to *obtain* the dependency differs.